SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGCACT 1956)

ENATHUR,KANCHIPURAM-631561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Nam **U.UDAYKIRAN**

Reg. **11249A3**

Clas **IIIB.E.(CSE)**

Course Code: BCSF245P80

Course Name:DATABASE MANAGEMENT SYSTEM

SRI CHANDRASEKHARENDRA SARASWATHI VISWA
M AH AVID YALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGCACT 1956)

ENATHUR,KANCHIPURAM-631561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this the bonafide record of work done by Mr/Ms. U.UDAY KIRAN ;
with Reg. No: 11249A391 of III-B.E.CSE(BCSF245FP80)(DATA BASE MANAGEMENTY SYSTEM) during the academic year 2025 – 2026.

Station : Enathur
Date:

Staff-in-charge Head of the Department

Submitted for the Practical examination held on ;

Examiner-1 Examiner-2

INDEX

SL.NO	PROGRAMNAME	DATE	PAGE NO	SIGN
01	Creation of a database&writing SQL queries			
02	Performing Insertion, Deletion, Modification, Alteration, Updating and Viewing of Records Based on Conditions			
03	Practicing DCL and TCL Commands			
04	Creating a Database and Applying Various Constraints			
05	Performing Queries Using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION and INTERSECT			
06	Queries Using Aggregate Functions, GROUP BY and HAVING Clauses			
07	Creation of Views, Sequences, Indexes and Using SAVEPOINT			
08	Normalization in Oracle (1NF, 2NF, 3NF, 4NF and 5NF) Using Func t ional D epe nde nci e s			
09	Creating Relationships Between Databases and Performing Joins and Subqueries			
10(a)	PL/SQL Block to Satisfy Conditions by Accepting Input from the User			
10(b)	PL/SQL Block for Handling Different Types of Exceptions			
10(c)	Creation of Procedures and Functions			
10(d)	Creation of Database Triggers - INSERT, DELETE and UPDATE Triggers			
11(a)	Database Design for Student Information System			
11(b)	Database Design for Hospital Management System			
11(c)	Database Design for Railway Reservation System			
11(d)	Database Design for Timetable Management System			

11(e)	Database Design for Hotel Management System			

EXPERIMENT – 1(A) **DATE:**

Creation of a database&writing SQL queries

A IM

To create a database table in Oracle and perform SQL queries to insert records and retrieve information from the database.

ALGORITHM

1. Start the Oracle SQL environment.
2. Create a table named STUDENT with suitable fields.
3. Define appropriate data types and a primary key.
4. Insert student records into the table.
5. Display all the records using the SELECT command.
6. Retrieve specific records using the WHERE condition.
7. Retrieve selected columns from the table.
8. Sort the records using the ORDER BY clause.
9. Display the required results.
10. Stop.

PROGRAM / SQL CODE

Command 1 - Create the STUDENT table

```
CREATE TABLE STUDENT (
    STUDENT_ID NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(30),
    DEPARTMENT VARCHAR2(20),
    AGE NUMBER(2),
    MARKS NUMBER(5,2)
) ;
```

Output:

Table created.

Command 2 - Insert STUDENT record

```
INSERT INTO STUDENT VALUES (1 01,'ARUN', 'CSE', 20, 85);
```

Output:

1 row created.

Command 3 - Insert STUDENT record

```
INSERT INTO STUDENT VALUES ( 102,'PRIYA', 'ECE', 21, 78);
```

Output:

1 row created.

Command 4 - Insert STUDENT record

```
INSERT INTO STUDENT VALUES ( 103, 'RAHUL', 'CSE', 20, 92);
```

Output:

1 row created.

Command 5 - Insert STUDENT record

```
INSERT INTO STUDENT VALUES (1 04, 'DIVYA', 'IT', 21, 88);
```

Output:

1 row created.

Command 6 - Insert STUDENT record

```
INSERT INTO STUDENT VALUES (10 5,'KARTHIK', 'CSE', 22, 75);
```

Output:

1 row created.

Command 7 - Commit the records

```
COM MIT;
```

Output:

Commit complete.

Command 8 - View all records

```
SELECT * FROM STUDENT;
```

Output:

STUDENT_ID	NAME	DEPARTMENT	AGE	MARKS
101	ARUN	CSE	20	85
102	PRIYA	ECE	21	78
103	RAHUL	CSE	20	92
104	DIVYA	IT	21	88
105	KARTHIK	CSE	22	75

Command 9 - Retrieve CSE students

```
SELECT * FROMSTUDENT
WHERE DEPARTMENT = 'CSE';
```

Output:

STUDENT_ID	NAME	DEPARTMENT	AGE	MARKS
101	ARUN	CSE	20	85
103	RAHUL	CSE	20	92
105	KARTHIK	CSE	22	75

Command 10 - Retrieve NAME and MARKS above 80

```
SELECT NAME, MARKS FROMSTUDENT
WHERE MARKS > 80;
```

Output:

NAME	MAR KS
ARUN	85
RAHUL	92
DIV YA	88

Command 11 - Retrieve selected columns

```
SELECT STUDENT_ID, NAME,
DEPARTMENT FROM STUDENT;
```

Output:

STUDENT_ID	NAME	DEPARTMENT
101	ARUN	CS
102	PRIYA	E
103	RAHUL	EC
104	DIVYA	E
105	KARTHIK	CS
		E
		IT
		CS
		E

Command 12 - Sort records by MARKS descending

```
SELECT NAME,  DEPARTMENT, MARKS FROM
STUDENT ORDER BY MARKS DESC;
```

Output:

NAME	DEPARTMENT	MARKS
RAHUL	CSE	92
DIVYA	IT	88
ARUN	CSE	85
PRIYA	ECE	78
KARTHIK	CSE	75

FINAL RESULT

Thus, the STUDENT database table was successfully created, records wereinserted, and SQL queries were successfully executed to retrieve the required information based on different conditions.

EXPERIMENT–1(B) **DATE:**

Creation of a database and writing SQL queries to retrieve information from the Database.

AIM

To create a database table in Oracle and perform SQL queries to insert records and retrieve information from the database.

ALGORITHM

1. Start the Oracle SQL environment.
2. Create a table named STUDENT with suitable fields.
3. Define appropriate data types and a primary key.
4. Insert student records into the table.
5. Display all the records using the SELECT command.
6. Retrieve specific records using the WHERE condition.
7. Retrieve selected columns from the table.
8. Sort the records using the ORDER BY clause.
9. Display the required results.
10. Stop.

PROGRAM / SQL CODE

Command 1 - Create the table

```
SQL>CREATE TABLE STUDENT (
                STUDENT_ID  NUMBER(5) PRIMARY KEY,
               NAME VARCHAR2(30),
               DEP ARTME NT  VARCHAR2(2 0),
                AGE NUMBER(2),
                MAR KS NUM BER (5, 2)
        );
```

Result:

Table created.

Command 2 - Insert record 1

```
SQL>INSERT  INTO  STUDENT  VA LUES  (1 01, 'ARUN', 'CSE', 20, 85);
```

Resu lt:

1 row created.

Command 3 - Insert record 2

SQL>INSERT INTO STUDENT VALUES (102, 'PRIYA', 'ECE', 21, 78);

Resu lt:

1 rowcreated.

Command 4 - Insert record 3

SQL>INSERT INTO STUDENT VALUE S (103, 'RAHUL', 'CSE', 20, 92);

Resu lt:

1 row created.

Command 5 - Insert record 4

SQL>INSERT INTO STUDENT VA LUES (1 04, 'DIVYA', 'IT', 21, 88);

Resu lt:

1 row created.

Command 6 - Insert record 5

SQL>INSERT INTO STUDENT V ALUES (105 , 'KARTHIK', 'CSE', 22, 75);

Resu lt:

1 row created.

Command 7 - Save the changes

SQL>COMMI T;

Result:

Commit complete.

.

Query 1 - Display all student records

```
SQL>SELECT     *   FROM   STUDENT;
```

Resu lt:

STUDENT_ID	NAME	DEPARTMENT	AGE	MA RKS
101	ARUN	CSE	20	85
102	PRIYA	ECE	21	78
103	RAHUL	CSE	20	92
104	DIVYA	IT	21	88
105	KARTHIK	CSE	22	75

Query 2 - Display only CSE students

```
SQL>SELECT *   FROM STUDENT
        WHERE  DEPARTMENT   =’CSE’
```

Result:

STUDENT_ID	NAME	DEPARTMENT	AGE	MARKS
101	ARUN	CSE	20	85
103	RAHUL	CSE	20	92
105	KARTHIK	CSE	22	75

Query 3 - Display students who scored more than 80

```
SQL>SELECT NAME, MARKS  FROM STUDENT
        WHERE MARKS>80;
```

Resu lt:

NAME	MARKS
ARUN	85
RAHUL	92
DIVYA	88

Query 4 - Display selected columns

```
SQL>SELECT STUDENT_ID, NAME, DEPARTMENT   FROM  STUDE NT;
```

Result:

STUDENT_ID	NAME	DEPARTMENT
101	ARUN	CSE
102	PRIYA	ECE
103	RAHUL	CSE
104	DIVYA	IT
105	KARTHIK	CSE

Query 4 - Display selected IN descending order of marks

```
SQL>SELECT STUDENT_ID, NAME, DEPARTMENT,MARKS   FROM STUDENT
            ORDER  BY  MARKS DESC;
```

Result:

NAME	DE PARTM ENT	MARKS
RAHUL	CSE	92
DIVYA	IT	88
ARUN	CSE	85
PRI YA	ECE	78
KAR THIK	CSE	75

4 . FINAL RESULT

Thus, the STUDENT database table was successfully created, records were inserted, and SQL queries were successfully executed to retrieve the required information based on different conditions.

EXPERIMENT–2 **DATE:**

Performing insertion, Deletion, Modifying, Altering, Updating and Viewing records based on conditions.

AIM

To perform insertion, deletion, modification, alteration, updating and viewing ofrecords based on specified conditions using SQL commands.

ALGORITHM

1. Create the STUDENT_LIST table.
2. Insert the given student records.
3. View all records.
4. Insert a new record.
5. Update the inserted record.
6. Delete the inserted record.
7. Alter the table structure.
8. Viewrecords based on a condition.

PROGRAM / SQL CODE

Command 1 – Create the table

```
SQL>CREATE TABLE STUDENT_LIST
   (
    NAME VARCHAR2(20),
    REG_NO NUMBER(5),
    PHONE_NUM NUMBER(10),
    SUBJECT VARCHAR2(30)
   );
```

Result:

Table created.

Command 2 – Insert ARYA

```
SQL>INSERT INTO STUDENT_LIST  VALUES
          ('arya', 23651, 9782775762, 'ECE');
```

.

Result:

1 row created.

Command 3 – Insert ASHNA

```
SQL>INSERT INTO STUDENT_LIST  VALUES
        ('ashna', 23761, 9786545762, 'CSE');
```

Result:

1 row created.

Command 4 – Insert HARIKA

```
SQL>INSERT INTO STUDENT_LIST VALUES
        ('harika', 43761, 9786564562, 'CSE');
```

Result:

1 row created.

Command 5 – View all records

```
SQL>SELECT  *  FROM  STUDE NT_LI ST;
```

Resu lt:

NAME	REG_NO	PHONE_NUM	SUBJECT
Arya	23651	9782775762	ECE
Ashna	23761	9786545762	CSE
harika	43761	9786564562	CSE

Command 6 – Insert a temporary record

```
SQL>INSERT INTO STUDENT_LIST VALUES
        ('uday', 12345, 9876543210, 'AI&ML ');
```

Result:

1 row created.

Command 7 – View the inserted record

```
SQL>SELECT * FROM STUDENT_LIST
        WHERE  REG_NO = 12345;
```

Result:

NAME	REG_NO	PHONE_NUM	SUBJECT
UDAY	12345	9876543210	AI&ML

.

Command 8 – Update the record

```
SQL>UPDATE STUDENT_LIST
        SET SUBJECT ='CSE'
        WHERE REG_NO= 12345 ;
```

Result:

1 row updated.

Command 9 – View the updated record

```
SQL>SELECT * FROM STUDENT_LIST
      WHERE REG_NO= 12345;
```

Result:

```
NAME   REG_NO   PHONE_NUM    SUBJ ECT
------ -------  ------------ ---- ----
UDAY   12345    9876543210     CSE
```

Command 10 – Delete the temporary record

```
SQL>DELETE FROM STUDENT_LIST
        WHERE  REG_NO =12345;
```

Result:

1 row deleted.

Command 11 – Alter the table

```
SQL>ALTER TABLE STUDENT_LIST MODIFY
        (SUBJECT  VARCHAR2(40));
```

Result:

Table altered.

Command 12 – View CSE students

```
SQL>SELECT * FROM STUDENT_LIST
        WHERE  SUBJECT  =' CSE';
```

Result:

```
NAME   REG_NO  PHONE_NUM  SUBJ ECT
------ -------  ---------- ---- ----
ashna  23761    9786545762    CSE
harika 43761    9786564562    CSE
```

FINAL RESULT

Thus, the required insertion, deletion, modification, alteration, updating andviewing operations were successfully executed.

EXPERIMENT – 3 **DATE:**

Practicing DCL&TCL Commands

AIM

To practice and execute DCL and TCL commands using the ACCOUNT and FLOWER_INVENTORY tables.

ALGORITHM

1. Create the ACCOUNT table.
2. Create the FLOWER_INVENTORY table.
3. Insert the given records.
4. Display the records.
5. Perform an update operation.
6. Create a savepoint.
7. Perform another update.
8. Roll back to the savepoint.
9. Commit the transaction.
10. Execute GRANT and REVOKE commands.

PROGRAM / SQL CODE

Command 1 – Create ACCOUNT

```
SQL>CREATE TABLE ACCOUNT (
        USER_ID NUMBER PRIMARY KEY,
        USER_NAME VARCHAR2(50),
        FOLLOWERS NUMBER
      );
```

Result:

Table created.

Command 2 – Create FLOWER_INVENTORY

```
SQL>CREATE TABLE FLOWER_INVENTORY  (
        FLOWER_ID NUMBER PRIMARY KEY,
        COMMON_NAME VARCHAR2(100) NOT NULL,
        SCIENTIFIC_NAME VARCHAR2(100), 6
        COLOR VARCHAR2(30) NOT NULL,
        PRICE_PER_STEM NUMBER(5,2),
        STOCK_QUANTITY NUMBER
      );
```

Result:

Table created.

Command 3 – Insert ACCOUNT record

```
SQL>INSERT INTO ACCOUNT
        VALUES (28748, 'uday', 181);
```

Result:

1 row created.

Command 4 – Insert ACCOUNT record

```
SQL>INSERT INTO ACCOUNT
        VAL UES (92723, 'harika', 231);
```

Result:

1 row created.

Command 5 – Insert ACCOUNT record 3

```
SQL>INSERT INTO ACCOUNT
        VALUES (64723, 'raya', 567);
```

Result:

1 row created.

Command 6 – Insert FLOWER_INVENTORY record 1

```
SQL>INSERT INTO FLOWER_INVENTORY
        VALUES (1, 'Red Rose', 'Rosa rubiginosa', 'Red', 4.5, 50);
```

Result:

1 row created.

Command 7 – Insert FLOWER_INVENTORY record 3

```
SQL>INSERT INTO FLOWER_INVENTORY
        VALUES (3, 'Sunflower', 'Helianthus annuus', 'Yellow', 3.5, 75);
```

Result:

1 row created.

Command 8 – Insert FLOWER_INVENTORY record 4

```
SQL>INSERT INTO FLOWER_INVENTORY
`       VALUES (4, 'Tulip', 'Tulipa gesneriana', 'Red', 5, 40);
```

Result:

1 row created.

Command 9 – Insert FLOWER_INVENTORY record 5

```
SQL>INSERT INTO FLOWER_INVENTORY
        VALUES  (5, 'Orchid', 'Orchidaceae', 'Purple', 8.5, 20);
```

Result:

1 rowcreated.

Command 10 – View ACCOUNT

SQL>SELECT * FROMACCOU NT;

Result:

USER_ID	USER_NAME	FOLLOWERS
28748	uday	181
92723	harika	231
64723	raya	567

Command 11 – View FLOWER_INVENTORY

SQL>SELECT * FROM FLOWE R_INV ENTOR Y;

Resu lt:

FLOWER_ID STOCK_QUANTITY

FLOWER_ID	COMMON_NAME	SCIENTIFIC_NAME	COLOR	PRICE_PER_STEM	STOCK_QUANTITY
1	Red Rose	Rosa rubiginosa	Red	4.5	50
3	Sunflower	Helianthus annuus	Yellow	3.5	75
4	Tulip	Tulipa gesneriana	Red	5	40
5	Orchid	Orchidaceae	Purple	8.5	20

Command 12 – Update Uday followers

SQL>UPDATE ACCOUNT SET FOLLOWERS = 200
WHERE USER_NAME ='uday ';

Result:

1 row updated.

Command 13 – Create Savepoint S1

SQL >SAVE POINT TO S1;

Resu lt:

Savepoint created.

Command 14 – Update Uday followers again

```
SQL>UPDATE ACCOUNT  SET FOLLOWERS = 250
          WHERE USER_NAME = 'uday';
```

Result:

row updated.

Command 15 – Rollback to S1

```
SQL>ROLLBACK       S1;
                  TO
```

Result:

Rollback complete.

Command 16 – Check the value after rollback

```
SQL>SELECT * FROM ACCOUNT
        WHERE   USER_NAME  '=uday' ;
```

Result:

```
USER_ID   USER_NAME  FOLLOWER
----------   ------------    ------------
28748        uday             200
```

Command 17 – Commit

```
SQL> COMMI T;
```

Result:

Commit complete.

Command 18 – GRANT

```
SQL>GRANT SELECT ON ACCOUNT TO PUBLIC;
```

Resu lt:

Grant succeeded.

Command 19 – REVOKE

```
SQL>REVOKE SELECT ON ACCOUNT
FROM PUBLIC;
```

Resu lt:

Revoke succeeded.

FINAL RESULT

Thus, DCL commands GRANT and REVOKE and TCL commands SAVEPOINT, ROLLBACK and COMMIT were successfully practiced and executed.

EXPERIMENT – 4 **DATE:**

Creating a database to set various constraints.

AIM

To create tables and implement various constraints such as PRIMARY KEY NOT NULL, UNIQUE and CHECK.

ALGORITHM

1. Create the LIBRARY_MEMBER table.
2. Apply various constraints.
3. Create the EMPLOYEES table.
4. Apply various constraints.
5. Insert the given records.
6. Display the records.
7. Test the constraints using invalid data.

PROGRAM / SQL CODE

Command 1 – Create LIBRARY_MEMBER with constraints

```
SQL>CREATE TABLE LIBRARY_MEMBER  (
        MEMBER_ID NUMBER PRIMARY KEY,
        FIRST_NAME VARCHAR2(50) NOT NULL,
        LAST_NAME VARCHAR2(50) NOT NULL,
        AGE NUMBER CHECK (AGE>= 18),
        GENDER CHAR(1) CHECK (GENDER IN ('M','F')),
        EMAIL VARCHAR2(100) NOT NULL UNIQUE,
        PHONE VARCHAR2(15)
        );
```

Resu lt:

Table created.

Command 2 – Create EMPLOYEES with constraints

```
SQL>CREATE TABLE EMPLOYEES  (
        EMP_ID NUMBER PRIMARY KEY,
        EMP_NAME VARCHAR2(50) NOT NULL,
        SALARY NUMBER CHECK (SALARY>0)
      );
```

Result:

Table created.

Command 3 – Insert LIBRARY_MEMBER record 1

```
SQL>INSERT INTO LIBRARY_MEMBER VALUES
        (1, 'John',  'Doe', 25, 'M', 'john.doe@email.com',
                '1234 56789 0');
```

Result:

1 row created.

Command 4 – Insert LIBRARY_MEMBER record 2

```
SQL>INSERT INTO LIBRARY_MEMBER VALUES (2, 'Jane', 'Smith',
22, 'F', 3 'jane@email.com', '0987654321');
```

Result:

1 row created.

Command 5 – Insert EMPLOYEES record 1

```
SQL>INSERT INTO EMPLOYEES VALUES
        (1,  'Rahul',30000) ;
```

Result:

1 row created.

Command 6 – Insert EMPLOYEES record 2

```
SQL>INSERT INTO EMPLOYEES VALUES
        (2,  'Amit',40000) ;
```

Result:

1 row created.

Command 7 – Insert EMPLOYEES record 3

```
SQL>INSERT INTO EMPLOYEES  VALUES
        (3,  'ram',50000) ;
```

Result:

1 row created.

Command 8 – Insert EMPLOYEES record 4

```
SQL>INSERT INTO EMPLOYEES VALUES
        (4, 'bala',90000) ;
```

Result:

1 row created.

Command 9 – Insert EMPLOYEES record 5

```
SQL>INSERT INTO EMPLOYEES  VALUES
        (38467, 'Neha',30);
```

Result:

1 row created.

Command 10 – Insert EMPLOYEES record 6

```
SQL>INSERT INTO EMPLOYEES  VALUES
        (81734, 'Priya',180);
```

Result:

1 rowcreated.

Command 11 – Display LIBRARY_MEMBER

SQL>SELECT * FROM LIBRA RY_ME MBER;

Resu lt:

MEMBER_ID	FIRST_NAME	LA ST _ NA ME	AGE	GENDER	EMAIL
1	John	Doe	25	M	john.doe@email.com
2	Jane	Smith	22	F	jane@email.com

PHONE_NO
1234567890
0987654321

Command 12 – Display EMPLOYEES

SQL>SELECT * FROMEMPLO YEES;

Resu lt:

EMP_ID	EMP_NAME	SALARY
1	Rahul	30000
2	Amit	40000
3	ram	50000
4	bala	90000
38467	Neha	30
81734	Priya	180

Command 13 – Test PRIMARY KEY constraint

```
SQL>INSERT INTO EMPLOYEES  VALUES
        (1, 'Kiran',25000 );
```

Resu lt:

ORA-00001: unique constraint violate

Command 14 – Test CHECK constraint

```
SQL>INSERT INTO EMPLOYEES VALUES
                (10,  'Test', 5000 );
```

Resu lt:

ORA-02290: check constraint violated

4 . FINAL RESULT

Thus, various constraints including PRIMARY KEY, NOT NULL, UNIQUE and CHECK were successfully implemented and verified.

EXPERIMENT–5 **DATE:**

Performing Queries using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT, Constraints etc.

AIM

To perform queries using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT and constraints.

ALGORITHM

1. Create the DEPARTMENTS table.
2. Create the STUDENTS table.
3. Insert the given records.
4. Display the tables.
5. Perform the IN query.
6. Perform the ANY query.
7. Perform the ALL query.
8. Perform the EXISTS query.
9. Perform the NOT EXISTS query.
10. Perform the UNION query.
11. Perform the INTERSECT query.

PROGRAM / SQL CODE

Command 1 – Create DEPARTMENTS

```
SQL>CREATE TABLE DEPARTMENTS (
        DEPARTMENT_ID  NUMBER(38)  PRIMARY  KEY,
        DEPARTMENT_NAME VARCHAR2(50) NOT NULL
      );
```

Resu lt:

Table created.

Command 2 – Create STUDENTS

```
SQL>CREATE TABLE STUDENTS  (
        STUDENT_ID NUMBER PRIMARY KEY,
        STUDENT_NAME VARCHAR2(50) NOT NULL,
        DEPARTMENT_ID NUMBER,
        MARKS NUMBER
    );
```

Result:

Table created.

Command 3 – Insert DEPARTMENTS record 1

```
SQL>INSERT INTO DEPARTMENTS VALUES
                (10, 'Computer Science');
```

Resu lt:

1 row created.

Command 4 – Insert DEPARTMENTS record 2

```
SQL>INSERT INTO DEPARTMENTS   VALUES
          (20,    'Information Techno logy' );
```

Result:

1 row created.

Command 5 – Insert DEPARTMENTS record 3

```
SQL>INSERT INTO DEPARTMENTS  VALUES
           (30, 'Elect ronic s');
```

Result:

1 row created.

Command 6 – Insert DEPARTMENTS record 4

```
SQL>INSERT INTO DEPARTMENTS VALUES
           (40, 'Mecha nical ');
```

Result:

1 row created.

Command 7 – Insert STUDENTS record 1

```
SQL>INSERT INTO STUDENTS VALUES (1, 'Arun', 185);
```

Result:

1 row created.

Command 8 – Insert STUDENTS record 2

```
SQL>INSERT INTO STUDENTS  VALUES (2, 'Bala', 172);
```

Result:

1 row created.

Command 9 – Insert STUDENTS record 3

SQL>INSERT INTO STUDENTS VALUES (3, 'Chitra', 20, 91);

Resu lt:

row created.

Command 10 – Insert STUDENTS record 4

SQL>INSERT INTO STUDENTS VALUES (4, 'Divya', 20, 68);

Resu lt:

row created.

Command 11 – Insert STUDENTS record 5

SQL>INSERT INTO STUDENTS 2VALUES (5, 'Ezhil', 30, 78);

Resu lt:

row created.

Command 12 – Insert STUDENTS record 6

SQL>INSERT INTO STUDENTS VALUES (6, 'Farhan', 30, 95);

Result:

row created.

Command 13 – Display DEPARTMENTS

SQL>SELECT * FROM DEPAR TMENT S;

Resu lt:

DEPARTMENT_ID	DEPARTMENT_NAME
10	Computer Science
20	Information Technology
30	Electronics
40	Mechanical

Command 14 – Display STUDENTS

SQL>SELECT * FROM STUDE NTS;

Resu lt:

STUDENT_ID	STUDENT_NAME	DEPARTMENT_ID	MARKS
1	Arun	10	85
2	Bala	10	72
3	Chitra	20	91
4	Divya	20	68
5	Ezhil	30	78
6	Farhan	30	95

Command 15 – IN operator

```
SQL>SELECT STUDENT_NAME, MARKS  FROM STUDENTS
            WHERE   DEPARTMENT_ID    IN (10,2 0);
```

Result:

STUDENT_NAME	MARKS
Arun	85
Bala	72
Chitra	91
Divya	68

Command 16 – ANY operator

```
SQL>SELECT STUDENT_NAME, MARKS FROM STUDENTS
WHERE MARKS>ANY  (
          SELECT MARKS
          FROM STUDENTS
          WHERE DEPARTMENT_ID = 20
        );
```

Resu lt:

STUDENT_NAME	MARKS
Arun	85
Bala	72
Chitra	91
Ezhil	78
Farhan	95

Department 20 has marks 91 and 68. The ANY condition is true when the mark is greater than at least one of those values.

.

Command 17 – ALL operator

```
SQL>SELECT STUDENT_NAME, MARKS  FROM STUDENTS
WHERE MARKS>ALL (
            SELECT MARKS
            FROM STUDENTS
            WHERE DEPARTMENT_ID = 20
          );
```

Resu lt:

```
STUDENT_NAME    MARKS
----------------    --------
     Farhan              95
```

Command 18 – EXISTS operator

```
SQL>SELECT DEPARTMENT_NAME FROM DEPARTMENTS_ID
              WHERE EXISTS  (
                SELECT *
                FROM STUDENTS S
                WHERE S.DEPARTMENT_ID = D.DEPARTMENT_ID
              );
```

Resu lt:

```
DEPARTMENT_NAME
----------------------
Computer Science
Information Technology
Electronics
```

Command 19 – NOT EXISTS operator

```
SQL>SELECT DEPARTMENT_NAME FROM DEPARTMENTS_ID
WHERE NOT EXISTS (
     SELECT *   FROM STUDENTS S
     WHERES DEPARTMENT_ID = D.DEPARTMENT_ID
  );
```

Resu lt:

```
DEPARTMENT_NA ME
-------------------------
  Mechanical
```

.

Command 20 – UNION operator

```
SQL>SELECT   STUDENT_NAME     FROM   STUDENTS
WHERE MARKS>90
UNION
SELECT  STUDENT_NAME     FROM   STUDENTS
WHERE   DEPARTMENT_ID = 10;
```

Resu lt:

```
STU DENT_ NAME
------------------
  Arun
  Bala
  Chi tra
  Far han
```

Command 21 – INTERSECT operator

```
SQL>SELECT  STUDENT_NAME   FROM    STUDENTS
        WHERE DEPARTMENT_ID = 10 4 INTERSECT
        SELECT STUDENT_NAME   FROM  STUDENTS
        WHERE MARKS>80;
```

Result:

```
STUDENT_ NAME
--------------------
      Arun
```

4 . FINAL RESULT

Thus, the required SQL queries using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION,INTERSECT and constraints were successfully executed.

.

EXPERIMENT – 6 **DATE:**

Queries Using Aggregate Functions, GROUP BY and HAVING Clauses

1 . AIM

To create an Oracle database table and perform aggregate functions such as COUNT(), AVG(), MAX() and MIN() along with GROUP BY and HAVING clauses.

2 . ALGORITHM

1. Start the Oracle SQL*Plus environment.
2. Create the FACULTY table with faculty ID, name, department, salary and experience.
3. Insert sample faculty records.
4. Display all records using SELECT.
5. Use COUNT(), AVG(), MAX() and MIN() aggregate functions.
6. Group faculty records by department using GROUP BY.
7. Filter grouped results using HAVING.
8. Display the results.
9. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the FACULTY table

```
SQL> CREATE TABLE FACULTY (
        FACULTY_ID NUMBER PRIMARY KEY,
        FACULTY_NAME VARCHAR2(50),
        DEPARTMENT VARCHAR2(30),
        SALARY NUMBER,
        EXPERIENCE NUMBER
    );
```

Output:

```
Table created.
```

Command 2 - Insert sample records

```
SQL> INSERT INTO FACULTY VALUES
```

```
  (1,'RAVI','CSE',50000,5);
INSERT INTO FACULTY VALUES
  (2,'PRIYA','CSE',60000,7);
INSERT INTO FACULTY VALUES
  (3,'ARUN','ECE',45000,4);
INSERT INTO FACULTY VALUES
  (4,'MEENA','ECE',55000,6);
INSERT INTO FACULTY VALUES
  (5,'KUMAR','CSE',70000,10);
INSERT INTO FACULTY VALUES
  (6,'SITA','EEE',40000,3);
INSERT INTO FACULTY VALUES
  (7,'RAJ','EEE',48000,5);
INSERT INTO FACULTY VALUES
  (8,'ANU','ECE',65000,8);
```

Output:

```
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
```

Command 3 - Display all records

```
SQL>SELECT*FROM FACULTY;
```

Output:

```
FACULTY_ID  FACULTY_NAME  DEPARTMENT  SALARY  EXPERIENCE
_______  ________  _______  _____  _______
```

1	RAVI	CSE	50000	5
2	PRIYA	CSE	60000	7
3	ARUN	ECE	45000	4
4	MEENA	ECE	55000	6
5	KUMAR	CSE	70000	10
6	SITA	EEE	40000	3
7	RAJ	EEE	48000	5
8	ANU	ECE	65000	8

Command 4 - COUNT()

```
SQL> SELECT COUNT(*) AS TOTAL_FACULTY
    FROM FACULTY;
```

Output:

```
TOTAL_FACULTY
--------
8
```

Command 5 - AVG()

```
SQL> SELECT AVG(SALARY) AS AVERAGE_SALARY
    FROM FACULTY;
```

Output:

```
AVERAGE_SALARY
---------
54125
```

Command 6 - MAX()

```
SQL> SELECT MAX(SALARY) AS HIGHEST_SALARY
    FROM FACULTY;
```

Output:

```
HIGHEST_SALARY
---------
70000
```

Command 7 - MIN()

```
SQL> SELECT MIN(SALARY) AS LOWEST_SALARY
    FROM FACULTY;
```

Output:

```
LOWEST_SALARY
-------------
40000
```

Command 8 - GROUP BY: number of faculty in each department

```
SQL> SELECT DEPARTMENT, COUNT(*) AS TOTAL_FACULTY
    FROM FACULTY
    GROUP BY DEPARTMENT;
```

Output:

```
DEPARTMENT   TOTAL_FACULTY
----------   -------------
CSE          3
ECE          3
EEE          2
```

Command 9 - GROUP BY: average salary by department

```
SQL> SELECT DEPARTMENT, AVG(SALARY) AS AVG_SALARY
    FROM FACULTY
    GROUP BY DEPARTMENT;
```

Output:

```
DEPARTMENT  AVG_SALARY
----------  ----------
CSE         60000
ECE         55000
EEE         44000
```

Command 10 - HAVING: departments with more than 2 faculty

```
SQL> SELECT DEPARTMENT, COUNT(*) AS TOTAL_FACULTY
```

```
    FROM FACULTY
    GROUP BY DEPARTMENT
    HAVING COUNT(*) > 2;
```

Output:

```
DEPARTMENT  TOTAL_FACULTY
----------  -------------
CSE         3
ECE         3
```

Command 11 - HAVING: average salary greater than 50000

```
SQL> SELECT DEPARTMENT, AVG(SALARY) AS AVG_SALARY
    FROM FACULTY
    GROUP BY DEPARTMENT
    HAVING AVG(SALARY) > 50000;
```

Output:

```
DEPARTMENT  AVG_SALARY
----------  ----------
CSE         60000
ECE         55000
```

4 . FINAL RESULT

Thus, the FACULTY table was created successfully and aggregate functions, GROUP BY and HAVING clauses were executed successfully in Oracle.

EXPERIMENT – 7 **DATE:**

Creation of Views, Sequences, Indexes and Using SAVEPOINT

1 . AIM

To study and execute SQL commands for creating and manipulating views, sequences, indexes and savepoints using Oracle SQL.

2 . ALGORITHM

1. Start Oracle SQL*Plus.
2. Create a sample EMPLOYEE table.
3. Insert sample records into the table.
4. Create a view using CREATE VIEW.
5. Create a sequence using CREATE SEQUENCE and use NEXTVAL.
6. Create an index using CREATE INDEX.
7. Create a SAVEPOINT, perform a rollback to it and commit the transaction.
8. Display and verify the results.
9. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the EMPLOYEE table

```
SQL> CREATE TABLE EMPLOYEE (
        EMP_ID NUMBER PRIMARY KEY,
        EMP_NAME VARCHAR2(30),
        SALARY NUMBER(10,2),
        DEPARTMENT VARCHAR2(20)
    );
```

Output:

```
Table created.
```

Command 2 - Insert sample records

```
SQL> INSERT INTO EMPLOYEE
        VALUES (101, 'JOHN', 30000, 'IT');
    INSERT INTO EMPLOYEE
```

```
    VALUES (102, 'MARY', 40000, 'IT');
INSERT INTO EMPLOYEE
    VALUES (103, 'MAY', 40000, 'IT');
INSERT INTO EMPLOYEE
    VALUES (104, 'RAVI', 45000, 'SALES');
```

Output:

1 row created.

1 row created.

1 row created.

Command 3 - Save the changes

```
SQL> COMMIT;
```

Output:

Commit complete.

Command 4 - Display the table

```
SQL> SELECT * FROM EMPLOYEE;
```

Output:

```
EMP_ID  EMP_NAME  SALARY  DEPARTMENT
------  --------  ------  ----------
101     JOHN      30000   IT
102     MARY      40000   IT
103     MAY       40000   IT
104     RAVI      45000   SALES
```

Command 5 - Create a view

```
SQL> CREATE VIEW EMPLOYEE_VIEW AS
    SELECT EMP_ID, EMP_NAME, SALARY
    FROM EMPLOYEE
    WHERE DEPARTMENT = 'IT';
```

Output:

View created.

Command 6 - Display the view

```
SQL> SELECT * FROM EMPLOYEE_VIEW;
```

Output:

```
EMP_ID  EMP_NAME  SALARY
------  --------  ------
101     JOHN      30000
102     MARY      40000
103     MAY       40000
```

Command 7 - Create a sequence

```
SQL> CREATE SEQUENCE EMPLOYEE_SEQ
    START WITH 105
    INCREMENT BY 1;
```

Output:

Sequence created.

Command 8 - Use NEXTVAL

```
SQL> SELECT EMPLOYEE_SEQ.NEXTVAL
    FROM DUAL;
```

Output:

```
NEXTVAL
-------
105
```

Command 9 - Insert a new employee using the sequence

```
SQL> INSERT INTO EMPLOYEE
      VALUES (EMPLOYEE_SEQ.NEXTVAL,
      'KUMAR',38000,'IT');
```

Output:

1 row created.

Command 10 - Commit

```
SQL> COMMIT;
```

Output:

```
Commit complete.
```

Command 11 - Create an index

```
SQL> CREATE INDEX EMPLOYEE_NAME_IDX
    ON EMPLOYEE (EMP_NAME);
```

Output:

```
Index created.
```

Command 12 - View the indexes on EMPLOYEE

```
SQL> SELECT INDEX_NAME, TABLE_NAME
    FROM USER_INDEXES
    WHERE TABLE_NAME = 'EMPLOYEE';
```

Output:

```
INDEX_NAME        TABLE_NAME
----------  -------
SYS_C...          EMPLOYEE
EMPLOYEE_NAME_IDX  EMPLOYEE
```

Command 13 - Insert a record before the savepoint

```
SQL> INSERT INTO EMPLOYEE
    VALUES (200, 'AVON', 25000, 'HR');
```

Output:

```
1 row created.
```

Command 14 - Create the savepoint S1

```
SQL> SAVEPOINT S1;
```

Output:

```
Savepoint created.
```

Command 15 - Update the salary

```
SQL> UPDATE EMPLOYEE
    SET SALARY = 28000
    WHERE EMP_ID = 200;
```

Output:

1 row updated.

Command 16 - Rollback to the savepoint

```
SQL> ROLLBACK TO S1;
```

Output:

Rollback complete.

Command 17 - Commit

```
SQL> COMMIT;
```

Output:

Commit complete.

4 . FINAL RESULT

Thus, views, sequences, indexes and SAVEPOINT operations were created and executed successfully in Oracle SQL.

EXPERIMENT – 8 **DATE:**

Normalization in Oracle (1NF, 2NF, 3NF, 4NF and 5NF) Using Functional Dependencies

1 . AIM

To study and implement database normalization in Oracle SQL by converting relations into 1NF, 2NF, 3NF, 4NF and 5NF to reduce redundancy and improve data integrity.

2 . ALGORITHM

1. Create tables containing student, course, department, employee, hobby, language, supplier, part and project details as required.
2. Identify functional dependencies and candidate keys.
3. Convert the relation to 1NF by ensuring atomic values.
4. Convert to 2NF by removing partial dependencies.
5. Convert to 3NF by removing transitive dependencies.
6. Convert to 4NF by separating non-trivial multivalued dependencies.
7. Convert to 5NF by decomposing relations based on join dependencies.
8. Execute CREATE TABLE and INSERT statements in Oracle SQL.
9. Display the resulting tables using SELECT and stop.

3 . PROGRAM / SQL CODE

Command 1 - 1NF: Create the STUDENT_PHONE table

```
SQL> CREATE TABLE STUDENT_PHONE (
    STUDENT_ID NUMBER,
    STUDENT_NAME VARCHAR2(30),
    PHONE_NUMBER VARCHAR2(15),
    PRIMARY KEY (STUDENT_ID, PHONE_NUMBER)
  );
```

Output:

```
Table created.
```

Command 2 - 1NF: Insert records

```
SQL> INSERT INTO STUDENT_PHONE
```

```
    VALUES (101, 'ARUN', '9876543210');
INSERT INTO STUDENT_PHONE
    VALUES (101, 'ARUN', '8765432109');
INSERT INTO STUDENT_PHONE
    VALUES (102, 'PRIYA', '7654321098');
```

Output:

```
1 row created.
1 row created.
1 row created.
```

Command 3 - 1NF: View the table

```
SQL> SELECT * FROM STUDENT_PHONE;
```

Output:

```
STUDENT_ID  STUDENT_NAME  PHONE_NUMBER
----------  ------------  ------------
101         ARUN          9876543210
101         ARUN          8765432109
102         PRIYA         7654321098
```

Command 4 - 2NF: Create the STUDENTS table

```
SQL> CREATE TABLE STUDENTS (
    STUDENT_ID NUMBER PRIMARY KEY,
    STUDENT_NAME VARCHAR2(30)
);
```

Output:

```
Table created.
```

Command 5 - 2NF: Create the COURSES table

```
SQL> CREATE TABLE COURSES (
    COURSE_ID VARCHAR2(10) PRIMARY KEY,
    COURSE_NAME VARCHAR2(30)
);
```

Output:

Table created.

Command 6 - 2NF: Create the ENROLLMENT table

```
SQL> CREATE TABLE ENROLLMENT (
        STUDENT_ID NUMBER,
        COURSE_ID VARCHAR2(10),
        MARKS NUMBER,
        PRIMARY KEY (STUDENT_ID, COURSE_ID),
        FOREIGN KEY (STUDENT_ID)
          REFERENCES STUDENTS(STUDENT_ID),
       FOREIGN KEY (COURSE_ID)
          REFERENCES COURSES(COURSE_ID)
     );
```

Output:

Table created.

Command 7 - 2NF: Insert records

```
SQL> INSERT INTO STUDENTS VALUES (101, 'ARUN');
    INSERT INTO STUDENTS VALUES (102, 'PRIYA');
    INSERT INTO COURSES VALUES ('C01', 'ORACLE');
    INSERT INTO COURSES VALUES ('C02', 'JAVA');
    INSERT INTO ENROLLMENT VALUES (101, 'C01', 85);
    INSERT INTO ENROLLMENT VALUES (101, 'C02', 90);
    INSERT INTO ENROLLMENT VALUES (102, 'C01', 78);
```

Output:

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

Command 8 - 2NF: View the tables

SQL> SELECT * FROM STUDENTS;

Output:

STUDENT_ID STUDENT_NAME

---------- ------------

101   ARUN

102   PRIYA

Command 9 - 2NF: View COURSES

SQL> SELECT * FROM COURSES;

Output:

COURSE_ID COURSE_NAME

---------- ------------

C01   ORACLE

C02   JAVA

Command 10 - 2NF: View ENROLLMENT

SQL> SELECT * FROM ENROLLMENT;

Output:

STUDENT_ID COURSE_ID MARKS

---------- ---------- -----

101   C01   85

101   C02   90

102   C01   78

Command 11 - 3NF: Create the DEPARTMENTS table

```
SQL> CREATE TABLE DEPARTMENTS (
        DEPT_ID VARCHAR2(10) PRIMARY KEY,
        DEPT_NAME VARCHAR2(30)
```

```
);
```

Output:

Table created.

Command 12 - 3NF: Create the EMPLOYEES table

```
SQL> CREATE TABLE EMPLOYEES (
    EMP_ID NUMBER PRIMARY KEY,
    EMP_NAME VARCHAR2(30),
    DEPT_ID VARCHAR2(10),
    FOREIGN KEY (DEPT_ID)
      REFERENCES DEPARTMENTS(DEPT_ID)
  );
```

Output:

Table created.

Command 13 - 3NF: Insert records

```
SQL> INSERT INTO DEPARTMENTS VALUES ('D01', 'IT');
  INSERT INTO DEPARTMENTS VALUES ('D02','HR');
  INSERT INTO EMPLOYEES VALUES (1, 'ARUN', 'D01');
  INSERT INTO EMPLOYEES VALUES (2, 'PRIYA', 'D02');
  INSERT INTO EMPLOYEES VALUES (3, 'KUMAR', 'D01');
```

Output:

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

Command 14 - 3NF: View DEPARTMENTS

```
SQL> SELECT * FROM DEPARTMENTS;
```

Output:

DEPT_ID DEPT_NAME

---- ------

D01 IT

D02 HR

Command 15 - 3NF: View EMPLOYEES

```
SQL> SELECT * FROM EMPLOYEES;
```

Output:

```
EMP_ID  EMP_NAME  DEPT_ID
------  --------  -------
1       ARUN      D01
2       PRIYA     D02
3       KUMAR     D01
```

Command 16 - 4NF: Create the STUDENT_HOBBIES table

```
SQL> CREATE TABLE STUDENT_HOBBIES (
       STUDENT_ID NUMBER,
       HOBBY VARCHAR2(30),
       PRIMARY KEY (STUDENT_ID, HOBBY)
     );
```

Output:

```
Table created.
```

Command 17 - 4NF: Create the STUDENT_LANGUAGES table

```
SQL> CREATE TABLE STUDENT_LANGUAGES (
       STUDENT_ID NUMBER,
       LANGUAGE VARCHAR2(30),
       PRIMARY KEY (STUDENT_ID, LANGUAGE)
     );
```

Output:

```
Table created.
```

Command 18 - 4NF: Insert records

```
SQL> INSERT INTO STUDENT_HOBBIES
      VALUES (101, 'CRICKET');
    INSERT INTO STUDENT_HOBBIES
      VALUES (101, 'MUSIC');
    INSERT INTO STUDENT_LANGUAGES
      VALUES (101, 'TAMIL');
    INSERT INTO STUDENT_LANGUAGES
      VALUES (101, 'ENGLISH');
```

Output:

```
1 row created.
1 row created.
1 row created.
1 row created.
```

Command 19 - 4NF: View STUDENT_HOBBIES

```
SQL> SELECT * FROM STUDENT_HOBBIES;
```

Output:

```
STUDENT_ID  HOBBY
----------  -----
101         CRICKET
101         MUSIC
```

Command 20 - 4NF: View STUDENT_LANGUAGES

```
SQL> SELECT * FROM STUDENT_LANGUAGES;
```

Output:

```
STUDENT_ID  LANGUAGE
----------  --------
101         TAMIL
101         ENGLISH
```

Command 21 - 5NF: Create the SUPPLIER_PART table

```
SQL> CREATE TABLE SUPPLIER_PART (
```

```
    SUPPLIER VARCHAR2(10),
    PART VARCHAR2(10),
    PRIMARY KEY (SUPPLIER, PART)
  );
```

Output:

Table created.

Command 22 - 5NF: Create the SUPPLIER_PROJECT table

```
SQL> CREATE TABLE SUPPLIER_PROJECT (
    SUPPLIER VARCHAR2(10),
    PROJECT VARCHAR2(10),
    PRIMARY KEY (SUPPLIER, PROJECT)
  );
```

Output:

Table created.

Command 23 - 5NF: Create the PART_PROJECT table

```
SQL> CREATE TABLE PART_PROJECT (
    PART VARCHAR2(10),
    PROJECT VARCHAR2(10),
    PRIMARY KEY (PART, PROJECT)
  );
```

Output:

Table created.

Command 24 - 5NF: Insert records

```
SQL> INSERT INTO SUPPLIER_PART VALUES ('S1', 'P1');
  INSERT INTO SUPPLIER_PART VALUES ('S2', 'P2');
  INSERT INTO SUPPLIER_PART VALUES ('S3', 'P1');
  INSERT INTO SUPPLIER_PROJECT
    VALUES ('S1', 'J1');
  INSERT INTO SUPPLIER_PROJECT
```

```
    VALUES ('S2', 'J2');
INSERT INTO PART_PROJECT VALUES ('P1', 'J1');
INSERT INTO PART_PROJECT VALUES ('P2', 'J1');
INSERT INTO PART_PROJECT VALUES ('P1', 'J2');
```

Output:

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

Command 25 - 5NF: View SUPPLIER_PART

```
SQL> SELECT * FROM SUPPLIER_PART;
```

Output:

```
SUPPLIER  PART
--------  ----
S1        P1
S2        P2
S3        P1
```

Command 26 - 5NF: View SUPPLIER_PROJECT

```
SQL> SELECT * FROM SUPPLIER_PROJECT;
```

Output:

```
SUPPLIER  PROJECT
--------  -------
S1        J1
S2        J2
```

Command 27 - 5NF: View PART_PROJECT

SQL> SELECT * FROM PART_PROJECT;

Output:

PART PROJECT

--- -----

P1 J1

P2 J1

P1 J2

4 . FINAL RESULT

Thus, normalization up to 5NF was implemented in Oracle SQL, reducing data redundancy and improving data integrity.

EXPERIMENT – 9 **DATE:**

Creating Relationships Between Databases and Performing Joins and Subqueries

1 . AIM

To create related tables using primary and foreign keys in Oracle and to perform joins and subqueries on them.

2 . ALGORITHM

1. Start the Oracle SQL environment.
2. Create the DEPARTMENT table with a primary key.
3. Create the EMPLOYEE table with a primary key and a foreign key referencing DEPARTMENT.
4. Insert records into both tables.
5. Test the foreign key constraint with an invalid record.
6. Perform inner join and left outer join queries.
7. Perform subqueries using IN, a comparison with an aggregate value and a correlated subquery.
8. Display the results.
9. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the DEPARTMENT table

```
SQL> CREATE TABLE DEPARTMENT (
    DEPT_ID NUMBER PRIMARY KEY,
    DEPT_NAME VARCHAR2(30) NOT NULL
  );
```

Output:

Table created.

Command 2 - Create the EMPLOYEE table with a foreign key

```
SQL> CREATE TABLE EMPLOYEE (
    EMP_ID NUMBER PRIMARY KEY,
    EMP_NAME VARCHAR2(30) NOT NULL,
```

```
    DEPT_ID NUMBER,
    SALARY NUMBER,
    FOREIGN KEY (DEPT_ID)
      REFERENCES DEPARTMENT(DEPT_ID)
  );
```

Output:

Table created.

Command 3 - Insert records

```
SQL> INSERT INTO DEPARTMENT
  VALUES (10, 'Computer Science');
INSERT INTO DEPARTMENT
  VALUES (20, 'Electronics');
INSERT INTO DEPARTMENT
  VALUES (30, 'Mechanical');
INSERT INTO EMPLOYEE
  VALUES (1, 'Meena', 10, 50000);
INSERT INTO EMPLOYEE
  VALUES (2, 'Raj', 10, 60000);
INSERT INTO EMPLOYEE
  VALUES (3, 'Priya', 20, 45000);
INSERT INTO EMPLOYEE
    VALUES (4, 'Vijay', 20, 70000);
  COMMIT;
```

Output:

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

Commit complete.

Command 4 - Test the foreign key constraint

SQL> INSERT INTO EMPLOYEE

VALUES (5,'Test', 99, 1000);

Output:

ORA-02291: integrity constraint violated -

parent key not found

The record is rejected because DEPT_ID 99

does not exist in DEPARTMENT.

Command 5 - Display the DEPARTMENT table

SQL> SELECT * FROM DEPARTMENT;

Output:

DEPT_ID	DEPT_NAME
10	Computer Science
20	Electronics
30	Mechanical

Command 6 - Display the EMPLOYEE table

SQL> SELECT * FROM EMPLOYEE;

Output:

EMP_ID	EMP_NAME	DEPT_ID	SALARY
1	Meena	10	50000
2	Raj	10	60000
3	Priya	20	45000
4	Vijay	20	70000

Command 7 - Inner join

```
SQL> SELECT E.EMP_NAME, D.DEPT_NAME, E.SALARY
     FROM EMPLOYEE E
     JOIN DEPARTMENT D
     ON E.DEPT_ID = D.DEPT_ID
     ORDER BY E.EMP_ID;
```

Output:

```
EMP_NAME   DEPT_NAME          SALARY
_____   ___________   _____
Meena      Computer Science   50000
Raj        Computer Science   60000
Priya      Electronics        45000
Vijay      Electronics        70000
```

Command 8 - Left outer join

```
SQL> SELECT D.DEPT_NAME, E.EMP_NAME
     FROM DEPARTMENT D
     LEFT OUTER JOIN EMPLOYEE E
     ON D.DEPT_ID = E.DEPT_ID
     ORDER BY D.DEPT_ID, E.EMP_ID;
```

Output:

```
DEPT_NAME          EMP_NAME
__________   ______
Computer Science   Meena
Computer Science   Raj
Electronics        Priya
Electronics        Vijay
Mechanical
```

Command 9 - Subquery using IN

```
SQL> SELECT EMP_NAME
```

```
    FROM EMPLOYEE
    WHERE DEPT_ID IN
      (SELECT DEPT_ID FROM DEPARTMENT
       WHERE DEPT_NAME = 'Computer Science');
```

Output:

```
EMP_NAME
------
Meena
Raj
```

Command 10 - Subquery using an aggregate value

```
SQL> SELECT EMP_NAME, SALARY
    FROM EMPLOYEE
    WHERE SALARY >
      (SELECT AVG(SALARY) FROM EMPLOYEE);
```

Output:

```
EMP_NAME  SALARY
------  -----
Raj       60000
Vijay     70000
```

Command 11 - Correlated subquery

```
SQL> SELECT E.EMP_NAME, E.SALARY
    FROM EMPLOYEE E
    WHERE E.SALARY >
      (SELECT AVG(S.SALARY)
       FROM EMPLOYEE S
       WHERE S.DEPT_ID = E.DEPT_ID)
    ORDER BY E.EMP_ID;
```

Output:

```
EMP_NAME  SALARY
```

------ -----

Raj 60000

Vijay 70000

4 . FINAL RESULT

Thus, related tables were created with a primary key and a foreign key, and joins and subqueries were executed successfully in Oracle.

EXPERIMENT – 10(a) **DATE:**

PL/SQL Block to Satisfy Conditions by Accepting Input from the User

1 . AIM

To write a PL/SQL block that accepts input from the user and checks conditions using IF-ELSE statements.

2 . ALGORITHM

1. Start.
2. Accept a number from the user.
3. Check whether the number is positive, negative, or zero.
4. Display the result.
5. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Enable server output

```
SQL> SET SERVEROUTPUT ON;
```

Command 2 - PL/SQL block to check the number

```
SQL> DECLARE
       n NUMBER;
     BEGIN
       n := &n;
       IF n > 0 THEN
         DBMS_OUTPUT.PUT_LINE('The number is positive');
       ELSIF n < 0 THEN
         DBMS_OUTPUT.PUT_LINE('The number is negative');
       ELSE
         DBMS_OUTPUT.PUT_LINE('The number is zero');
       END IF;
     END;
     /
```

Output:

Enter value for n: 10
The number is positive

PL/SQL procedure successfully completed.

4 . FINAL RESULT

Thus, a PL/SQL block was successfully executed by accepting user input and checking conditions using IF-ELSE statements.

EXPERIMENT – 10(b) **DATE:**

PL/SQL Block for Handling Different Types of Exceptions

1 . AIM

To write and execute a PL/SQL block to handle different types of exceptions.

2 . ALGORITHM

1.Start.
2.Declare variables and accept input from the user.
3.Execute statements that may cause exceptions.
4.Handle predefined exceptions using the EXCEPTION block.
5.Handle other errors using WHEN OTHERS.
6.Display the appropriate message.
7.Stop.

3 . PROGRAM / SQL CODE

Command 1 - Enable server output

```
SQL> SET SERVEROUTPUT ON;
```

Command 2 - PL/SQL block with exception handling

```
SQL> DECLARE
      a NUMBER;
      b NUMBER;
      c NUMBER;
    BEGIN
      a := &a;
      b := &b;
      c := a / b;
      DBMS_OUTPUT.PUT_LINE('Result = ' || c);
    EXCEPTION
      WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Error: Division by zero');
      WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Error: Invalid value');
      WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: some other exception occurred');
    END;
    /
```

Output:

Enter value for a: 10

Enter value for b: 2

Result = 5

PL/SQL procedure successfully completed.

Command 3 - Other exception run

Output:

Error: some other exception occurred.

PL/SQL procedure successfully completed.

4 . FINAL RESULT

Thus, a PL/SQL block was successfully executed to handle predefined and other exceptions.

EXPERIMENT – 10(c) **DATE:**

Creation of Procedures and Functions

1 . AIM

To create and execute a procedure and a function in PL/SQL.

2 . ALGORITHM

1.Start.
2.Create a procedure to calculate the square of a number.
3.Execute the procedure by passing a number.
4.Create a function to calculate the square of a number.
5.Call the function and display the result.
6.Stop.

3 . PROGRAM / SQL CODE

Command 1 - Enable server output

```
SQL> SET SERVEROUTPUT ON;
```

Command 2 - Create the procedure

```
SQL> CREATE OR REPLACE PROCEDURE
      find_square(n IN NUMBER) AS
    BEGIN
     DBMS_OUTPUT.PUT_LINE('Square = ' || (n*n));
    END;
    /
```

Output:

Procedure created.

Command 3 - Execute the procedure

```
SQL> EXEC find_square(5);
```

Output:

Square = 25

PL/SQL procedure successfully completed.

Command 4 - Create the function

```
SQL> CREATE OR REPLACE FUNCTION
      calculate_square(n IN NUMBER)
    RETURN NUMBER
    IS
    BEGIN
     RETURN n * n;
    END;
    /
```

Output:

```
Function created.
```

Command 5 - Call the function

```
SQL> BEGIN
      DBMS_OUTPUT.PUT_LINE('Square = ' ||
       calculate_square(5));
    END;
    /
```

Output:

```
Square = 25

PL/SQL procedure successfully completed.
```

4 . FINAL RESULT

Thus, a procedure and a function were successfully created and executed in Oracle PL/SQL.

EXPERIMENT – 10(d) **DATE:**

Creation of Database Triggers - INSERT, DELETE and UPDATE

1 . AIM

To create and execute INSERT, DELETE and UPDATE triggers in Oracle.

2 . ALGORITHM

1. Start.
2. Create an Employee table.
3. Create triggers to display a message after inserting a record, deleting a record and updating a record.
4. Execute the triggers by performing INSERT, DELETE and UPDATE operations.
5. Display the results.
6. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the Employee table

```
SQL> CREATE TABLE Employee (
        Emp_ID NUMBER PRIMARY KEY,
        Emp_Name VARCHAR2(30),
        Salary NUMBER
     );
```

Output:

```
Table created.
```

Command 2 - INSERT trigger

```
SQL> CREATE OR REPLACE TRIGGER insert_trigger
     AFTER INSERT ON Employee
     BEGIN
       DBMS_OUTPUT.PUT_LINE('Record inserted successfully');
     END;
     /
```

Output:

Trigger created.

Command 3 - DELETE trigger

```
SQL> CREATE OR REPLACE TRIGGER delete_trigger
    AFTER DELETE ON Employee
    BEGIN
      DBMS_OUTPUT.PUT_LINE('Record deleted successfully');
    END;
    /
```

Output:

Trigger created.

Command 4 - UPDATE trigger

```
SQL> CREATE OR REPLACE TRIGGER update_trigger
    AFTER UPDATE ON Employee
    BEGIN
      DBMS_OUTPUT.PUT_LINE('Record updated successfully');
    END;
    /
```

Output:

Trigger created.

Command 5 - Execute the INSERT trigger

```
SQL> SET SERVEROUTPUT ON;
     INSERT INTO Employee
       VALUES (1,'Ravi',3000);
```

Output:

Record inserted successfully

1 row created.

Command 6 - Execute the UPDATE trigger

```
SQL> UPDATE Employee
     SET Salary = 35000
     WHERE Emp_ID = 1;
```

Output:

Record updated successfully

1 row updated.

Command 7 - Execute the DELETE trigger

```
SQL> DELETE FROM Employee
     WHERE Emp_ID = 1;
```

Output:

Record deleted successfully

1 row deleted.

4 . FINAL RESULT

Thus, INSERT, DELETE and UPDATE triggers were successfully created and executed in Oracle.

EXPERIMENT – 10(e) **DATE:**

Creation of Database with Implicit and Explicit Cursors

1 . AIM

To create and execute PL/SQL programs using implicit and explicit cursors in Oracle.

2 . ALGORITHM

1.Start.
2.Create an Employee table and insert records.
3.Use an implicit cursor to update a record and display the number of rows affected.
4.Declare an explicit cursor to retrieve employee details.
5.Open the cursor, fetch the records and close the cursor.
6.Display the result.
7.Stop.

3 . PROGRAM / SQL CODE

Command 1 - Enable server output

```
SQL> SET SERVEROUTPUT ON;
```

Command 2 - Create the Employee table

```
SQL> CREATE TABLE Employee (
        Emp_ID NUMBER PRIMARY KEY,
        Emp_Name VARCHAR2(30),
        Salary NUMBER
    );
```

Output:

```
Table created.
```

Command 3 - Insert records

```
SQL> INSERT INTO Employee
        VALUES (1,'Ravi',30000);
    INSERT INTO Employee
```

```
    VALUES (2,'Sita',40000);
```

Output:

1 row created.

1 row created.

Command 4 - Implicit cursor

```
SQL> BEGIN
    UPDATE Employee
    SET Salary = Salary + 1000
    WHERE Emp_ID = 1;
     DBMS_OUTPUT.PUT_LINE('Rows updated: ' ||
      SQL%ROWCOUNT);
   END;
   /
```

Output:

Rows updated: 1

PL/SQL procedure successfully completed.

Command 5 - Explicit cursor

```
SQL> DECLARE
    CURSOR emp_cursor IS
     SELECT Emp_ID, Emp_Name, Salary
      FROM Employee;
   v_id Employee.Emp_ID%TYPE;
   v_name Employee.Emp_Name%TYPE;
   v_salary Employee.Salary%TYPE;
BEGIN
   OPEN emp_cursor;
   LOOP
     FETCH emp_cursor
```

```
        INTO v_id, v_name, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(v_id || ' ' ||
        v_name || ' ' || v_salary);
    END LOOP;
    CLOSE emp_cursor;
  END;
  /
```

Output:

1 Ravi 31000

2 Sita 40000

PL/SQL procedure successfully completed.

4 . FINAL RESULT

Thus, implicit and explicit cursors were successfully created and executed in Oracle PL/SQL.

EXPERIMENT – 11(a) **DATE:**

Database Design for Student Information System

1 . AIM

To design and implement a Student Information System database using Oracle to store and manage student details.

2 . ALGORITHM

1.Start.
2.Create a Student table.
3.Insert student details into the table.
4.Retrieve student records using SELECT.
5.Update student details using UPDATE.
6.Delete student records using DELETE.
7.Display the results.
8.Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the Student table

```
SQL> CREATE TABLE Student (
        Student_ID NUMBER PRIMARY KEY,
        Student_Name VARCHAR2(50),
        Age NUMBER,
        Gender VARCHAR2(10),
        Department VARCHAR2(30),
        Phone NUMBER(10),
        Email VARCHAR2(50)
    );
```

Output:

```
Table created.
```

Command 2 - Insert student details

```
SQL> INSERT INTO Student VALUES
```

```
(101,'Ravi',20,'male','CSE',
 9876543210,'ravi@gmail.com');
INSERT INTO Student VALUES
 (102,'Sita',19,'Female','ECE',
 9876543211,'sita@gmail.com');
INSERT INTO Student VALUES
 (103,'Ram',21,'male','IT',
 9876543212,'ram@gmail.com');
```

Output:

```
1 row created.
1 row created.
1 row created.
```

Command 3 - Retrieve student records

```
SQL> SELECT * FROM Student;
```

Output:

```
Student_ID Student_Name Age Gender Department Phone      Email
---------- ------------ --- ------ ---------- ---------- ---------------
101        Ravi         20  male   CSE        9876543210 ravi@gmail.com
102        Sita         19  Female ECE        9876543211 sita@gmail.com
103        Ram          21  male   IT         9876543212 ram@gmail.com
```

Command 4 - Update student department

```
SQL> UPDATE Student
    SET Department = 'CSE'
    WHERE Student_ID = 102;
```

Output:

```
1 row updated.
```

Command 5 - Commit and view the updated records

```
SQL> COMMIT;
    SELECT Student_ID, Student_Name,
     Department FROM Student;
```

Output:

Commit complete.

Student_ID	Student_Name	Department
101	Ravi	CSE
102	Sita	CSE
103	Ram	IT

Command 6 - Delete a student record

```
SQL> DELETE FROM Student
    WHERE Student_ID = 103;
```

Output:

1 row deleted.

Command 7 - Commit and view the final records

```
SQL> COMMIT;
    SELECT Student_ID, Student_Name,
     Department FROM Student;
```

Output:

Commit complete.

Student_ID	Student_Name	Department
101	Ravi	CSE
102	Sita	CSE

4 . FINAL RESULT

Thus, a Student Information System database was successfully designed and implemented using Oracle SQL.

EXPERIMENT – 11(b) **DATE:**

Database Design for Hospital Management System

1 . AIM

To design and implement a Hospital Management System using Oracle to store and manage patient, doctor and appointment details.

2 . ALGORITHM

1.Start.
2.Create the Patient, Doctor and Appointment tables.
3.Insert sample records into the tables.
4.Retrieve records using SELECT statements.
5.Update records.
6.Display the results.
7.Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the Patient table

```
SQL> CREATE TABLE Patient (
        Patient_ID NUMBER PRIMARY KEY,
        Patient_Name VARCHAR2(50),
        Age NUMBER,
        Gender VARCHAR2(10),
        Disease VARCHAR2(50),
        Phone NUMBER(10)
    );
```

Output:

```
Table created.
```

Command 2 - Create the Doctor table

```
SQL> CREATE TABLE Doctor (
        Doctor_ID NUMBER PRIMARY KEY,
        Doctor_Name VARCHAR2(50),
```

```
    Specialization VARCHAR2(30)
  );
```

Output:

```
Table created.
```

Command 3 - Create the Appointment table

```
SQL> CREATE TABLE Appointment (
    Appointment_ID NUMBER PRIMARY KEY,
    Patient_ID NUMBER
      REFERENCES Patient(Patient_ID),
    Doctor_ID NUMBER
      REFERENCES Doctor(Doctor_ID),
    Appointment_Date DATE
  );
```

Output:

```
Table created.
```

Command 4 - Insert sample records

```
SQL> INSERT INTO Patient VALUES
    (1,'Ravi',30,'male','Fever',
     9876543210);
  INSERT INTO Patient VALUES
    (2,'Sita',25,'Female','Cold',
     9876543211);
INSERT INTO Doctor VALUES
  (101,'Dr. Kumar','General Medicine');
INSERT INTO Doctor VALUES
  (102,'Dr. Paiyo','Cardiology');
INSERT INTO Appointment VALUES
  (1001,1,101,DATE'2026-11-01');
INSERT INTO Appointment VALUES
```

```
    (1002,2,102,DATE'2026-11-02');
COMMIT;
```

Output:

```
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
Commit complete.
```

Command 5 - Display the Patient table

```
SQL>SELECT*FROM Patient;
```

Output:

```
Patient_ID  Patient_Name  Age  Gender  Disease  Phone
----------  ------------  ---  ------  -------  ----------
1           Ravi          30   male    Fever    9876543210
2           Sita          25   Female  Cold     9876543211
```

Command 6 - Display the Doctor table

```
SQL>SELECT*FROM Doctor;
```

Output:

```
Doctor_ID  Doctor_Name  Specialization
---------  -----------  --------------
101        Dr. Kumar    General Medicine
102        Dr. Paiyo    Cardiology
```

Command 7 - Display appointment details

```
SQL>SELECT A.Appointment_ID,
      P.Patient_Name, D.Doctor_Name,
      A.Appointment_Date
    FROM Appointment A
```

```
    JOIN Patient P
      ON A.Patient_ID = P.Patient_ID
    JOIN Doctor D
      ON A.Doctor_ID = D.Doctor_ID;
```

Output:

```
Appointment_ID  Patient_Name  Doctor_Name  Appointment_Date
--------------  ------------  -----------  ----------------
1001            Ravi          Dr. Kumar    01-NOV-26
1002            Sita          Dr. Paiyo    02-NOV-26
```

Command 8 - Update patient details

```
SQL> UPDATE Patient
    SET Disease = 'Viral Fever'
    WHERE Patient_ID = 1;
    COMMIT;
```

Output:

```
1 row updated.

Commit complete.

Patient_ID  Patient_Name  Disease
----------  ------------  -----------
1           Ravi          Viral Fever
2           Sita          Cold
```

Command 9 - Delete an appointment

```
SQL> DELETE FROM Appointment
    WHERE Appointment_ID = 1002;
    COMMIT;
```

Output:

```
1 row deleted.
```

Commit complete.

Appointment_ID Patient_ID Doctor_ID
-------------- ---------- ---------
1001   1   101

4 . FINAL RESULT

Thus, a Hospital Management System database was successfully designed and implemented using Oracle SQL.

EXPERIMENT – 11(c) **DATE:**

Database Design for Railway Reservation System

1 . AIM

To design and implement a railway reservation database using Oracle.

2 . ALGORITHM

1. Start.
2. Create a Railway table with ticket number, passenger name, train name, source, destination and fare.
3. Insert passenger ticket details into the table.
4. Display all details using the SELECT statement.
5. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the Railway table

```
SQL> CREATE TABLE Railway (
        Ticket_NO INT PRIMARY KEY,
        Passenger_Name VARCHAR(30),
        Train_Name VARCHAR(30),
        Source VARCHAR(20),
        Destination VARCHAR(20),
        Fare INT
     );
```

Output:

Table created.

Command 2 - Insert ticket details

```
SQL> INSERT INTO Railway VALUES
        (101,'Ravi','Express','Chennai',
         'Hyderabad',500);
     INSERT INTO Railway VALUES
        (102,'Sita','Superfast',
```

'Vijayawada', 'Bangalore', 700);

Output:

1 row created.

1 row created.

Command 3 - Display all details

SQL> SELECT * FROM Railway;

Output:

Ticket_NO	Passenger_Name	Train_Name	Source	Destination	Fare
101	Ravi	Express	Chennai	Hyderabad	500
102	Sita	Superfast	Vijayawada	Bangalore	700

4 . FINAL RESULT

The railway reservation system table was created successfully, and passenger ticket details were inserted and displayed.

EXPERIMENT – 11(d) **DATE:**

Database Design for Timetable Management System

1 . AIM

To create a Timetable Management System using SQL to store and display timetable details.

2 . ALGORITHM

1. Start.
2. Create a timetable table with period number, day, subject, faculty and room number.
3. Insert timetable details into the table.
4. Display the details using the SELECT statement.
5. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the Timetable table

```
SQL> CREATE TABLE Timetable (
        Period_NO INT PRIMARY KEY,
        Day VARCHAR(15),
        Subject VARCHAR(30),
        Faculty VARCHAR(30),
        Room_NO INT
     );
```

Output:

```
Table created.
```

Command 2 - Insert timetable details

```
SQL> INSERT INTO Timetable VALUES
        (1, 'Monday', 'Java', 'Ravi', 101);
     INSERT INTO Timetable VALUES
        (2, 'Tuesday', 'SQL', 'Sita', 102);
```

Output:

1 row created.

1 row created.

Command 3 - Display the timetable

SQL> SELECT * FROM Timetable;

Output:

```
Period_NO  Day      Subject  Faculty  Room_NO
---------- -------- -------- -------- -------
1          Monday   Java     Ravi     101
2          Tuesday  SQL      Sita     102
```

4 . FINAL RESULT

Thus, the Timetable Management System was created successfully, and timetable details were inserted and displayed.

EXPERIMENT – 11(e) **DATE:**

Database Design for Hotel Management System

1 . AIM

To create a Hotel Management System using SQL to store and display guest and room details.

2 . ALGORITHM

1. Start.
2. Create a Hotel table with room number, guest name, room type, number of days and amount.
3. Insert guest details into the table.
4. Display the details.
5. Stop.

3 . PROGRAM / SQL CODE

Command 1 - Create the Hotel table

```
SQL> CREATE TABLE Hotel (
        Room_no INT PRIMARY KEY,
        Guest_Name VARCHAR(30),
        Room_Type VARCHAR(20),
        Days INT,
        Amount INT
     );
```

Output:

```
Table created.
```

Command 2 - Insert guest details

```
SQL> INSERT INTO Hotel VALUES
        (101, 'Ravi', 'AC', 2, 3000);
     INSERT INTO Hotel VALUES
        (102, 'Sita', 'Non-AC', 3, 2400);
```

Output:

1 row created.

1 row created.

Command 3 - Display the details

SQL> SELECT * FROM Hotel;

Output:

```
Room_no  Guest_Name  Room_Type  Days  Amount
-------  ----------  ---------  ----  ------
101      Ravi        AC         2     3000
102      Sita        Non-AC     3     2400
```

4 . FINAL RESULT

Thus, the Hotel Management System was created successfully, and guest details were inserted and displayed.